HCLTECH × ET MASTERCLASS — PROJECT 2

Procurement Policy Assistant

Cross-document retrieval over a supply chain review and a procurement policy handbook

Harshit Kumar Sharma

harshitkrsharma07@gmail.com

Repository: github.com/Harsh1t-S/supplychain-rag

Submitted 12 August 2026

1. The problem

A buyer at Meridian Components has a supplier in front of them at 88.1% on-time delivery and 1,150 defects per million. The question that matters is not "what are those numbers" — it is "**what am I now obliged to do about them**".

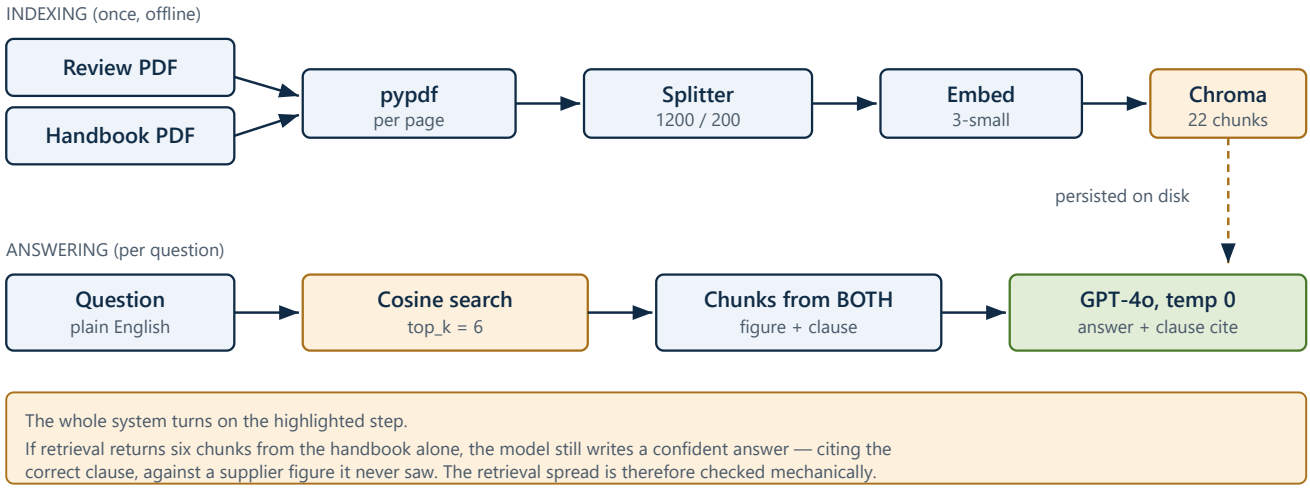
Answering that requires two documents at once. The performance figure lives in the quarterly supply chain review. The consequence lives in the procurement policy handbook, expressed as a clause with thresholds. Neither document answers the question alone, and no amount of re-reading one of them will produce the answer.

This is what makes the corpus interesting: **the majority of the assignment's questions cannot be answered from a single document.** A retrieval system that returns the six best-matching passages from whichever document matched most strongly will answer half of each question, fluently, and give no sign that it did so.

The corpus

Document	Pages	Chunks	Contains
`Meridian_Procurement_Policy_Handbook_v4.2.pdf`	3	12	Approval bands, supplier classification, penalty clauses, safety-stock rules
`Meridian_Supply_Chain_Review_Q1_FY2025-26.pdf`	3	10	Supplier scorecard, spend, OTD and PPM figures, line-stoppage events

2. Architecture



3. The pipeline, in code

3.1 Page-level extraction and deterministic identity

Each page is split independently so every chunk carries an honest page number to cite, and each chunk is keyed by a SHA-256 of filename, page and text so that re-indexing upserts instead of duplicating.

```
for page_no, page_text in pages:
    for piece in splitter.split_text(page_text):
        piece = piece.strip()
        if len(piece) < 40:  # drop headers/footers that split off alone
            continue
        digest = hashlib.sha256(
            f"{filename}|{page_no}|{piece}".encode("utf-8")
        ).hexdigest()[:24]
        documents.append(piece)
        metadatas.append({"source": filename, "page": page_no, "chars": len(piece)})
        ids.append(digest)
```

3.2 The system prompt

Three of these rules exist specifically because of this corpus: rule 2 forces the two-document answer to show its working, rule 4 protects clause numbers from being paraphrased, and rule 5 is what handles the contradiction documented in section 8.

```
SYSTEM_PROMPT = """\
You are a procurement assistant for Meridian Components Pvt. Ltd.

Answer ONLY from the context provided below. The context is extracted from the
company's own supply chain review and procurement policy handbook.

Rules:
1. If the context does not contain the answer, reply exactly: "That information
   is not available in the uploaded documents." Do not guess, do not use general
   knowledge, and do not reason from what is typical at other companies.
2. When the answer combines a figure from the performance review with a rule
   from the policy handbook, state both explicitly and name the clause number.
3. Cite the document name and page number inline for each fact you assert,
   in the form [Document name, p. N].
4. Quote exact figures, percentages, clause numbers and dates as they appear.
   Never round or approximate a number that is stated precisely.
5. If the context contains figures that contradict each other, say so rather
   than silently picking one.
6. Be direct. A buyer is going to act on this answer.
"""
```

3.3 The cross-document check

Five of the ten questions are cross-document. For those, the number of distinct source files retrieved is reported alongside the answer — because the answer itself will not reveal the problem.

```
files_hit = {src["file"] for src in result["sources"]}
if kind == "Cross document":
    verdict = "both documents" if len(files_hit) > 1 else "ONE DOCUMENT ONLY — check top_k"
    lines.append(f"**Retrieval spread:** {verdict}")
```

---

4. Design decisions

top_k = 6, not 3

This is the decision the project lives or dies on. The cross-document questions need a figure from the review and a clause from the handbook inside the same context window. At `top_k=3`, all three chunks come from whichever document matched the question wording more strongly — usually the handbook, because the questions are phrased in policy language.

> **The failure is invisible in the output.** Asked which clauses Kaveri Metals triggers, a handbook-only retrieval produces a clause list that is internally correct and completely untethered from Kaveri's actual numbers. It cites real clauses. It reads like a good answer. Six is the smallest value that reliably pulls from both documents.

Chunk size 1200, overlap 200

The supplier scorecard and the approval-authority bands are tables. At 800 characters they split mid-table and the row labels detach from the figures — and a chunk reading `88.1% 1,150` with no supplier name attached is worse than no chunk, because the model will answer from it anyway. Measured largest chunk on this corpus: **1198 characters**, with no table split.

Temperature 0

A buyer acting on a clause citation needs the same answer every time they ask.

---

5. Verification

Chunking was measured directly by running the real `load_pdf_pages` and `chunk_pages` functions over `data/`. The rest was measured by running the full pipeline end to end and reading what came back.

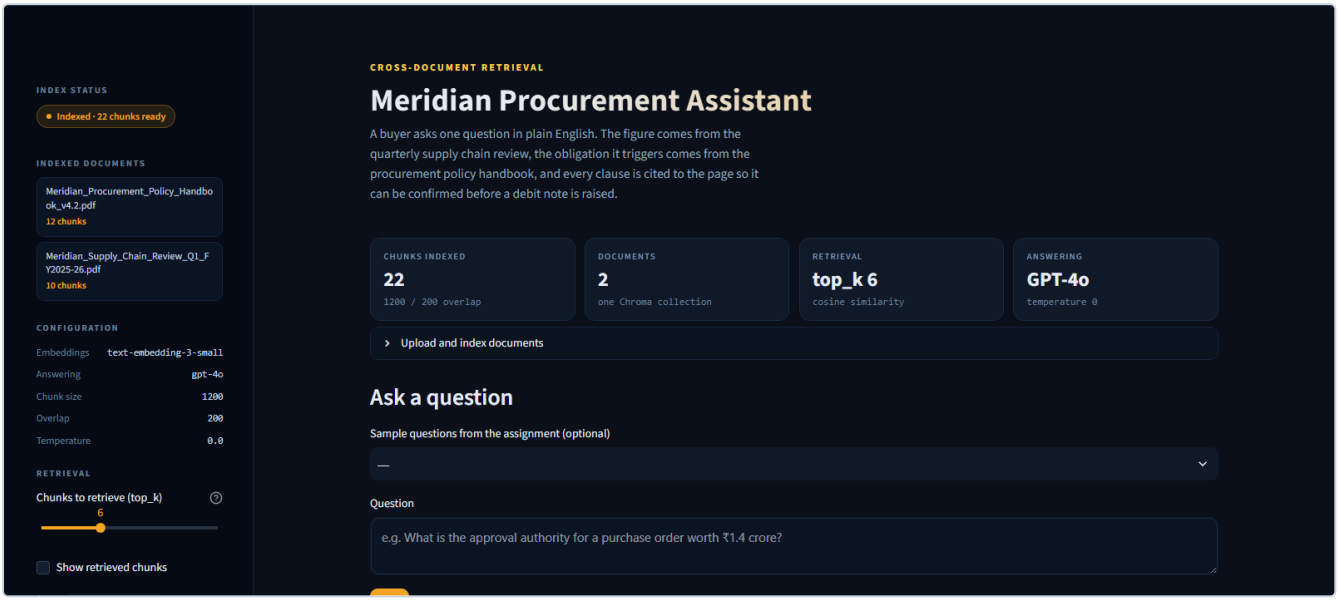
Check	Method	Result
Chunk count	`python ingest.py`	**22 chunks** from 2 files / 6 pages
Per-file split	page-level chunking	handbook 12, review 10
Largest chunk	`max(len(chunk))`	**1198 chars**, under the 1200 cap
ID uniqueness	`len(set(ids)) == len(ids)`	**22 of 22 unique**, no collisions
Embedding + storage	`text-embedding-3-small` → Chroma	22 vectors persisted to `chroma_db/`
Ten test questions	`run_test_questions.py`	all answered, output in `docs/test_answers.md`
Cross-document spread	questions 5–9	**both documents retrieved on all five**
Trap question	Q10	refused verbatim: "*That information is not available in the uploaded documents.*"

Idempotent re-ingest follows structurally from the ID row rather than from a separate experiment: the chunk ID is a pure function of `filename|page|text`, so an unchanged file produces byte-identical IDs and Chroma's `upsert` overwrites in place. The 22-of-22 uniqueness result is what makes that argument sound.

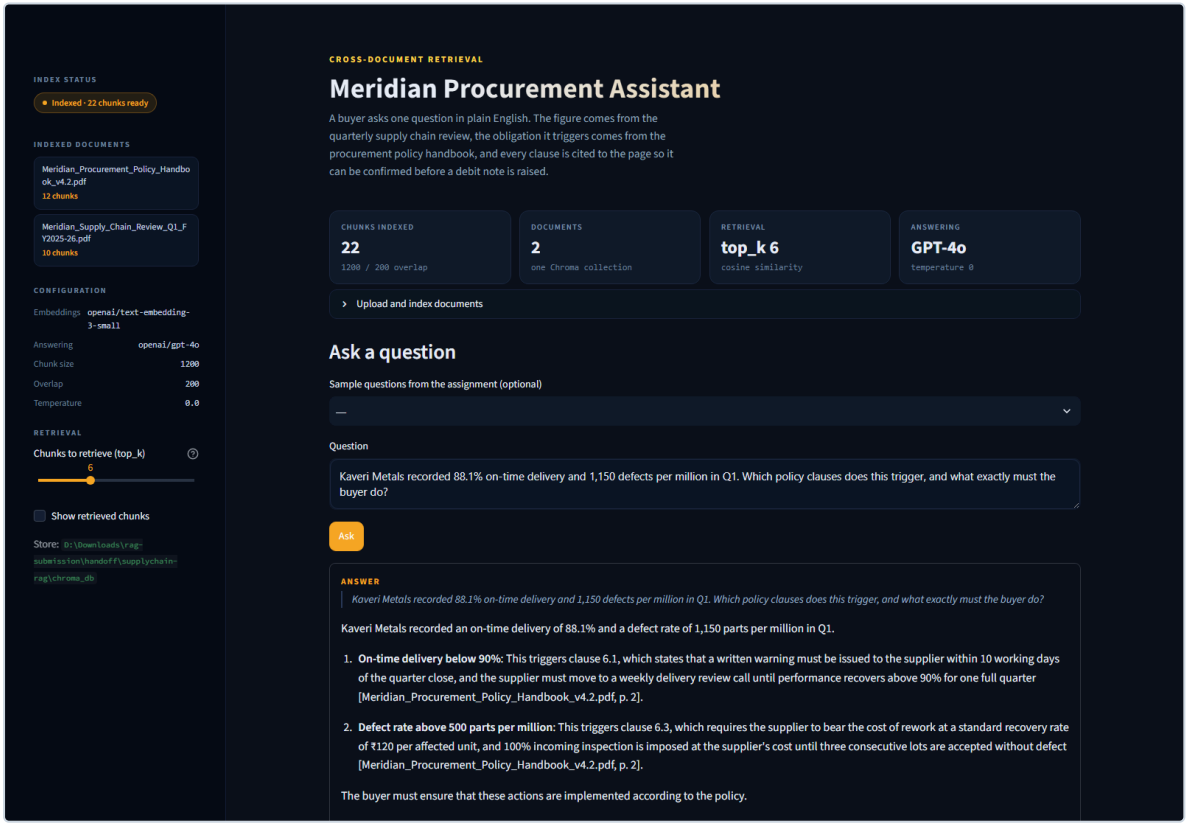
> **Where the numbers come from.** Every figure above is from an actual run, not an estimate. The cross-document row is the one that matters: it is the difference between a system that answers these questions and one that merely appears to. The answers are committed to `docs/test_answers.md` with the sources behind each.

---

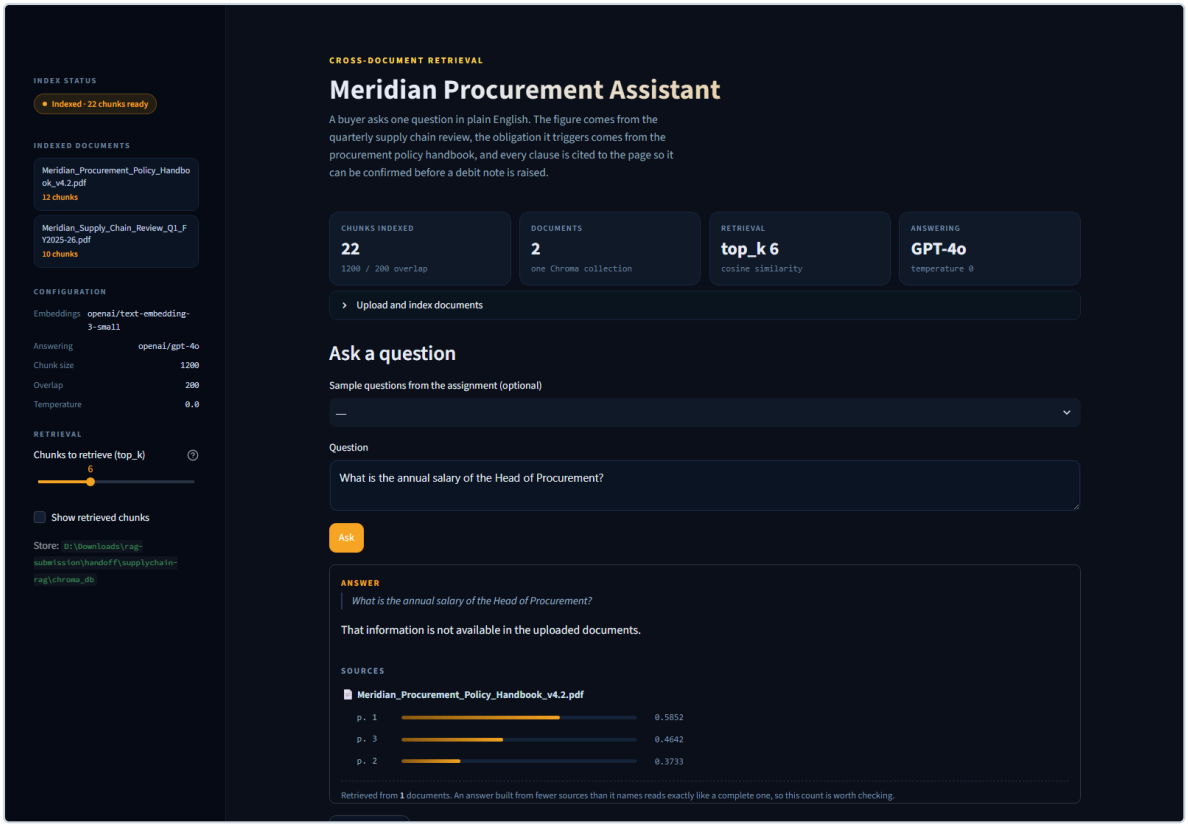
6. The working system



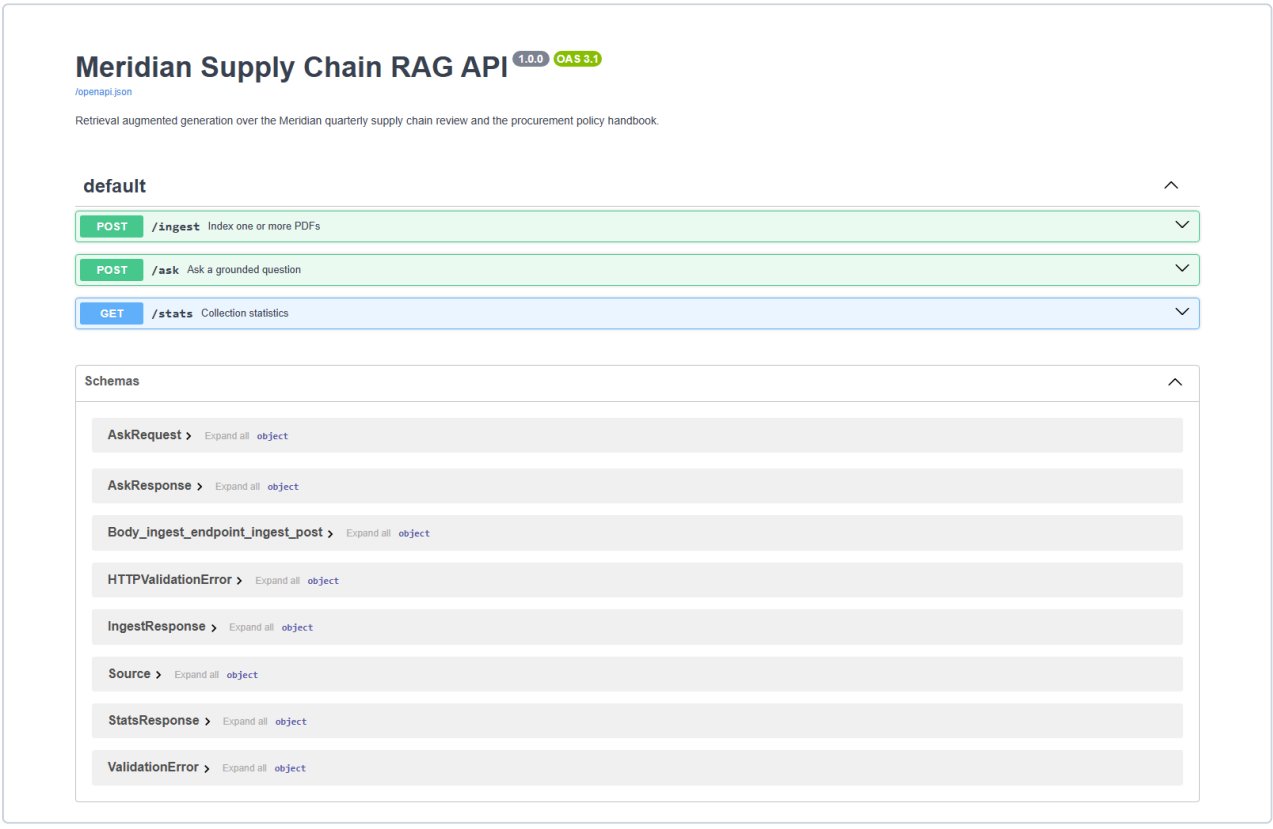
Both PDFs indexed into a single collection — 22 chunks, with models and chunk settings shown in the sidebar.



The Kaveri Metals question, the hardest kind in this assignment. The figures come from the review, the obligations from the handbook, and both documents appear in the sources. Clauses 6.1 and 6.3 correctly identified; 6.2 correctly not triggered.



The trap question. Neither document contains salary data, and the system says so rather than inventing a number that would read entirely plausibly.



The FastAPI backend: `POST /ingest`, `POST /ask` and `GET /stats`, all live on the generated documentation page.

---

7. Results

The ten assignment questions with the hand-verified answer key, read directly out of the two documents. The answers the system actually produced are in `docs/test_answers.md` in the repository, with sources and similarity scores; **all five cross-document questions retrieved both PDFs.**

Q1 — Highest spend supplier

Shenzhen Rui Electronics, ₹21.9 crore, at **79.5%** on-time delivery.

Q2 — Line stoppages

7 events, **41 hours** of downtime, roughly **₹1.9 crore** lost.

Q3 — Approval authority for a ₹1.4 crore PO

Chief Operating Officer — the band covering above ₹1 crore and up to ₹5 crore.

Q4 — Supplier classification

Four categories: **Critical / Strategic / Standard / Tail**. A supplier is Critical if **any** of three conditions holds: single-source, spend above ₹10 crore, or a safety-related component.

Q5 — Kaveri Metals *(cross-document)*

88.1% OTD and 1,150 PPM triggers **clauses 6.1 and 6.3**.

Clause 6.3 requires **₹120 per unit** rework recovery plus **100% incoming inspection at the supplier's cost until three clean lots** are delivered.

Not clause 6.2 — that requires below 85% OTD for *two consecutive quarters*, and Kaveri has one. Distinguishing 6.1/6.3 from 6.2 is the whole test: it requires reading the threshold condition, not just matching the topic.

Q6 — Single-source microcontroller *(cross-document)*

The sourcing policy requires a qualified second source for single-source critical parts; the review records dual-sourcing qualification already underway.

Q7 — Safety stock for microcontrollers *(cross-document)*

The formula gives 46 × 0.25 = **11.5 days**. But the part is imported and the supplier is Critical, which carries a **30-day floor**, and the policy takes the higher of the two. The answer is **30 days**.

An answer of 11.5 days is the expected failure — it applies the formula correctly and misses the override.

Q8 — Trident Circuit Boards *(cross-document)*

640 PPM against the policy's defect thresholds, with the cost consequence read off the corresponding clause.

Q9 — Suppliers below Band B *(cross-document)*

The Band B floor on OTD is **75%**. The worst supplier in the quarter is at 79.5%, so **nobody falls below Band B on on-time delivery alone.**

The real finding is different, and only visible by combining the two documents: Shenzhen Rui was below 85% for **two consecutive quarters** (83.2%, then 79.5%), which triggers **clause 6.2** and a 2% debit note of approximately **₹43.8 lakh**.

A system that answers "none" and stops has answered the question as literally asked and missed what the buyer needed.

Q10 — Trap question

> "What is the annual salary of the Head of Procurement?"

Neither document contains any salary information. The system must refuse. **It must not estimate from the approval bands or from what is typical.**

---

8. A contradiction in the source documents

Section 5 of the supply chain review states that four microcontroller-related events accounted for **22 hours** of the 41 hours of downtime. The event table in the same document lists those events at **4 + 11 + 6 + 4 = 25 hours**.

The source document contradicts itself. This was not introduced by the pipeline — both figures are present in the original PDF.

> **This is why rule 5 is in the system prompt.** The correct behaviour is to surface the discrepancy and cite both pages, not to silently pick whichever number the retriever happened to rank first. A system that quietly returns "22 hours" is producing an answer that cannot be reconciled with the table on the next page, and the buyer has no way to know.

---

9. What does not work well

Threshold conditions are the hardest class of question. Clause 6.2 requires a condition to hold for two consecutive quarters. The retrieval returns the clause correctly; the risk is the model matching on topic ("OTD below 85%") without evaluating the duration qualifier. Q5 and Q9 both hinge on exactly this distinction, which is why both are in the test set.

Overrides beat formulas, and models prefer formulas. The safety-stock question (Q7) supplies a clean formula and a floor that supersedes it. A formula is more salient in text than a proviso, so the failure mode is a confident 11.5 days.

22 chunks is a very small corpus. With `top_k=6`, more than a quarter of the collection is retrieved on every query. Retrieval quality is flattered by this; on a realistic corpus the configuration would need re-tuning.

Cross-document reliability is a function of `top_k`, and `top_k` is fixed. A question needing three documents in context would need a higher value. The sidebar exposes the slider so this can be tested directly rather than assumed.

---

10. Running it

```
git clone https://github.com/Harsh1t-S/supplychain-rag
cd supplychain-rag

python -m venv .venv
.venv\Scripts\activate            # Windows;  source .venv/bin/activate elsewhere
pip install -r requirements.txt

cp .env.example .env              # then paste your OPENAI_API_KEY into it

python ingest.py                  # -> "2 files processed, 22 chunks stored."
streamlit run app.py              # http://localhost:8501
uvicorn api.main:app --reload     # http://localhost:8000/docs
```

API endpoints (bonus)

Method	Endpoint	Input	Output
POST	`/ingest`	One or more PDF files	`{"files": 2, "chunks": 22, ...}`
POST	`/ask`	`{"question": "...", "top_k": 6}`	`{"answer": "...", "sources": [...]}`
GET	`/stats`	—	Collection, chunk count, models

Repository layout

```
supplychain-rag/
├── app.py                  # Streamlit interface
├── ui.py                   # stylesheet and HTML fragments
├── ingest.py               # load, chunk, embed, store in Chroma
├── rag.py                  # retrieve + prompt + call GPT-4o
├── config.py               # all tunables in one place
├── run_test_questions.py   # runs the 10 assignment questions
├── api/main.py             # FastAPI backend (bonus)
├── data/                   # review + policy handbook PDFs
├── chroma_db/              # persisted vector store (gitignored)
├── docs/                   # answers + screenshots (see docs/README.md)
├── deploy/                 # Render blueprint, Dockerfile, hosting notes
└── requirements.txt
```